

```

%%capture --no-display

import warnings
warnings.filterwarnings("ignore")

import pandas as pd
from collections import Counter
from IPython.display import display, Markdown
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import fpgrowth, association_rules

# =====
# DATA TRANSAKSI
# =====

transactions = [
    ['kopi', 'gula', 'susu'],
    ['kopi', 'roti'],
    ['kopi', 'gula'],
    ['teh', 'gula', 'roti'],
    ['kopi', 'susu'],
    ['teh', 'roti']
]

total_transaksi = len(transactions)
min_support_count = 2
min_support = min_support_count / total_transaksi

display(Markdown("# Latihan FP-Growth"))

data_transaksi = pd.DataFrame({
    'Transaksi': ['T1', 'T2', 'T3', 'T4', 'T5', 'T6'],
    'Item': [' ', '.join(item) for item in transactions]
})

display(Markdown("## Data Transaksi"))
display(data_transaksi)

# =====
# 1. HITUNG SUPPORT SETIAP ITEM
# =====

item_count = Counter()

for transaksi in transactions:
    for item in transaksi:
        item_count[item] += 1

support_df = pd.DataFrame({
    'Item': list(item_count.keys()),
    'Support Count': list(item_count.values()),
    'Support': [count / total_transaksi for count in item_count.values()],
    'Support (%)': [round((count / total_transaksi) * 100, 2) for count in item_count.values()]
})

support_df = support_df.sort_values(
    by=['Support Count', 'Item'],
    ascending=[False, True]
).reset_index(drop=True)

display(Markdown("## 1. Support Setiap Item"))
display(support_df)

# =====
# 2. FREQUENT ITEM MINIMUM SUPPORT = 2
# =====

frequent_items_df = support_df[support_df['Support Count'] >= min_support_count].copy()

display(Markdown("## 2. Frequent Item dengan Minimum Support = 2"))
display(frequent_items_df)

# =====
# 3. URUTKAN ITEM BERDASARKAN SUPPORT
# =====

urutan_item = list(frequent_items_df['Item'])

```

```

urutan_df = pd.DataFrame({
    'Urutan': range(1, len(urutan_item) + 1),
    'Item': urutan_item,
    'Support Count': list(frequent_items_df['Support Count'])
})

display(Markdown("## 3. Urutan Item Berdasarkan Support"))
display(urutan_df)

# =====
# TRANSAKSI SETELAH DIURUTKAN
# =====

ranking_item = {item: i for i, item in enumerate(urutan_item)}

transaksi_urut = []
for transaksi in transactions:
    item_urut = sorted(
        [item for item in transaksi if item in ranking_item],
        key=lambda item: ranking_item[item]
    )
    transaksi_urut.append(item_urut)

transaksi_urut_df = pd.DataFrame({
    'Transaksi': ['T1', 'T2', 'T3', 'T4', 'T5', 'T6'],
    'Item Setelah Diurutkan': [' ', '.join(item) for item in transaksi_urut]
})

display(Markdown("## Transaksi Setelah Diurutkan"))
display(transaksi_urut_df)

# =====
# 4. BENTUK FP-TREE SEDERHANA
# =====

class Node:
    def __init__(self, nama):
        self.nama = nama
        self.count = 0
        self.children = {}

root = Node("Root")

for transaksi in transaksi_urut:
    current = root
    for item in transaksi:
        if item not in current.children:
            current.children[item] = Node(item)
        current.children[item].count += 1
        current = current.children[item]

def tampilkan_tree(node, indent=""):
    hasil = ""
    children = list(node.children.values())
    for i, child in enumerate(children):
        connector = "└─ " if i == len(children) - 1 else "├─ "
        hasil += indent + connector + f"{child.nama}:{child.count}\n"
        next_indent = indent + ("  " if i == len(children) - 1 else "│  ")
        hasil += tampilkan_tree(child, next_indent)
    return hasil

fp_tree_text = "Root\n" + tampilkan_tree(root)

display(Markdown("## 4. FP-Tree Sederhana"))
display(Markdown(f"```\n{text}\n```"))

# =====
# 5. JALANKAN FP-GROWTH DENGAN PYTHON
# =====

te = TransactionEncoder()
data_encoded = te.fit(transactions).transform(transactions)

df = pd.DataFrame(data_encoded, columns=te.columns_)

display(Markdown("## One-Hot Encoding"))
display(df)

frequent_itemsets = fpgrowth(

```

```

df,
min_support=min_support,
use_colnames=True
)

frequent_itemsets['support_count'] = frequent_itemsets['support'] * total_transaksi

hasil_fpgrowth = frequent_itemsets.copy()
hasil_fpgrowth['itemsets'] = hasil_fpgrowth['itemsets'].apply(lambda x: ', '.join(list(x)))
hasil_fpgrowth['support_count'] = hasil_fpgrowth['support_count'].astype(int)

display(Markdown("## 5. Hasil FP-Growth"))
display(hasil_fpgrowth)

# =====
# 6. ASSOCIATION RULES DAN INTERPRETASI
# =====

try:
    rules = association_rules(
        frequent_itemsets,
        num_itemsets=total_transaksi,
        metric='confidence',
        min_threshold=0.5
    )
except TypeError:
    rules = association_rules(
        frequent_itemsets,
        metric='confidence',
        min_threshold=0.5
    )

hasil_rules = rules[['antecedents', 'consequents', 'support', 'confidence', 'lift']].copy()

hasil_rules['antecedents'] = hasil_rules['antecedents'].apply(lambda x: ', '.join(list(x)))
hasil_rules['consequents'] = hasil_rules['consequents'].apply(lambda x: ', '.join(list(x)))

hasil_rules = hasil_rules.sort_values(
    by=['confidence', 'lift'],
    ascending=[False, False]
).reset_index(drop=True)

display(Markdown("## 6. Association Rules"))
display(hasil_rules)

# =====
# INTERPRETASI MINIMAL 2 RULES
# =====

interpretasi_rules = pd.DataFrame({
    'Rule': ['teh → roti', 'susu → kopi'],
    'Interpretasi': [
        'Jika pelanggan membeli teh, maka pelanggan cenderung membeli roti. Rule ini kuat karena confidence 100% dan lift 1',
        'Jika pelanggan membeli susu, maka pelanggan cenderung membeli kopi. Rule ini kuat karena confidence 100% dan lift 1'
    ],
    'Kesimpulan': [
        'Produk teh dan roti dapat dipromosikan atau diletakkan berdekatan.',
        'Produk susu dan kopi dapat dijadikan paket promosi atau rekomendasi pembelian.'
    ]
})

display(Markdown("## Interpretasi Minimal 2 Association Rules"))
display(interpretasi_rules)

# =====
# KESIMPULAN
# =====

display(Markdown("""
## Kesimpulan

Berdasarkan hasil analisis, item yang paling sering muncul adalah kopi dengan support count sebanyak 4 kali.
Item yang termasuk frequent item adalah kopi, gula, roti, susu, dan teh karena semuanya memenuhi minimum support = 2.

Kombinasi item yang sering muncul bersama adalah kopi dengan gula, kopi dengan susu, dan roti dengan teh.

Association rules yang dapat digunakan adalah teh → roti dan susu → kopi.
Hasil ini dapat digunakan untuk membantu strategi promosi, rekomendasi produk, dan penataan barang di minimarket.
"""))

```


Latihan FP-Growth

Data Transaksi

	Transaksi	Item	
0	T1	kopi, gula, susu	
1	T2	kopi, roti	
2	T3	kopi, gula	
3	T4	teh, gula, roti	
4	T5	kopi, susu	
5	T6	teh, roti	

1. Support Setiap Item

	Item	Support Count	Support	Support (%)
0	kopi	4	0.666667	66.67
1	gula	3	0.500000	50.00
2	roti	3	0.500000	50.00
3	susu	2	0.333333	33.33
4	teh	2	0.333333	33.33

2. Frequent Item dengan Minimum Support = 2

	Item	Support Count	Support	Support (%)
0	kopi	4	0.666667	66.67
1	gula	3	0.500000	50.00
2	roti	3	0.500000	50.00
3	susu	2	0.333333	33.33
4	teh	2	0.333333	33.33

3. Urutan Item Berdasarkan Support

	Urutan	Item	Support Count
0	1	kopi	4
1	2	gula	3
2	3	roti	3
3	4	susu	2
4	5	teh	2

Transaksi Setelah Diurutkan

	Transaksi	Item Setelah Diurutkan
0	T1	kopi, gula, susu
1	T2	kopi, roti
2	T3	kopi, gula
3	T4	gula, roti, teh
4	T5	kopi, susu
5	T6	roti, teh

Langkah
berikutnya:

[Buat kode dengan data_transaksi](#)

[New interactive sheet](#)

[Buat kode dengan support_df](#)

[New interactive sheet](#)

[Buat k](#)

4. FP-Tree Sederhana

```
Root
├── kopi:4
│   ├── gula:2
│   │   └── susu:1
│   └── roti:1
│       └── susu:1
├── gula:1
│   └── roti:1
│       └── teh:1
└── roti:1
    └── teh:1
```

One-Hot Encoding

	gula	kopi	roti	susu	teh	
0	True	True	False	True	False	